

Tổng quan chương

Chương 1 đã phân tích các mục tiêu và giải pháp thiết kế tổng thể của hệ thống tương tác cử chỉ tay 3D thời gian thực. Để xây dựng hệ thống hoạt động ổn định và có độ chính xác cao trên các camera phổ thông, việc nghiên cứu các thuật toán nền tảng đóng vai trò quyết định. Chương 2 này sẽ trình bày chi tiết cơ sở lý thuyết khoa học của các thành phần trong hệ thống, bao gồm: (i) tiền xử lý luồng video trực tuyến với OpenCV; (ii) nguyên lý trích xuất 21 điểm mốc khớp bàn tay bằng mô hình học sâu MediaPipe Hands; (iii) các thuật toán chuẩn hóa tọa độ hình học và mô hình mạng nơ-ron truyền thẳng (MLP) cho phân loại cử chỉ tĩnh và động; (iv) cơ chế truyền nhận dữ liệu qua socket UDP; và (v) toán học của giải thuật lọc thích nghi chống rung One Euro Filter.

0.1 Tiền xử lý ảnh trực tuyến với OpenCV

Thư viện OpenCV (Open Source Computer Vision Library) **opencv_library** đóng vai trò là module thu nhận dữ liệu đầu vào. OpenCV giao tiếp trực tiếp với webcam thông qua lớp `cv2.VideoCapture` để nắm bắt các khung hình liên tiếp ở tần số cao (30 khung hình/giây). Các khung hình thu được từ camera thường ở định dạng màu sắc BGR (Blue-Green-Red) mặc định.

Trước khi đưa vào mô hình học sâu để xử lý, ảnh thô cần trải qua các bước tiền xử lý biến đổi hình học và không gian màu sắc: (i) lật ảnh theo chiều ngang (Horizontal Flip) bằng thuật toán biến đổi hình học trong OpenCV để lật khung hình theo trục dọc, đảm bảo chuyển động tự nhiên như nhìn gương; (ii) chuyển đổi không gian màu BGR sang RGB vì MediaPipe thường xử lý ảnh đầu vào theo định dạng RGB; và (iii) giảm độ phân giải khung hình (Downscaling) xuống mức HD (960×540) bằng bộ lọc nội suy tuyến tính để giảm tải dữ liệu pixel, hỗ trợ duy trì tốc độ xử lý khoảng trên 30 FPS trong điều kiện thử nghiệm, đồng thời vẫn giữ chất lượng định vị khớp tay ở mức chấp nhận được.

0.2 Trích xuất khớp tay dựa trên học sâu bằng MediaPipe Hands

Sau khi tiền xử lý ảnh đầu vào, hệ thống trích xuất đặc trưng khớp bàn tay bằng thư viện MediaPipe Hands của Google **lugaresi2019mediapipe**. Đây là giải pháp học sâu tích hợp sẵn, bao gồm hai mô hình mạng nơ-ron hoạt động nối tiếp nhau:

1. **Mạng phát hiện bàn tay (Hand Detector Model):** Sử dụng kiến trúc Single Shot Detector (SSD) được tối ưu hóa cho các thiết bị di động để nhanh chóng phát hiện vùng chứa bàn tay trong toàn bộ khung hình ảnh và xuất ra một khung bao giới hạn (bounding box).
2. **Mạng hồi quy điểm mốc bàn tay (Hand Landmark Model):** Chạy trên vùng

ảnh bàn tay đã được cắt ra từ mô hình phát hiện để dự đoán tọa độ 3D tương đối của 21 điểm mốc khớp bàn tay (landmarks). Sơ đồ 21 điểm mốc này được biểu diễn trong Hình 0.1.



Hình 0.1: Sơ đồ cấu trúc 21 điểm landmark khớp tay trong MediaPipe

Dữ liệu đầu ra của MediaPipe cho mỗi điểm mốc i ($i \in [0, 20]$) là một bộ ba tọa độ không gian (x_i, y_i, z_i) , trong đó: x_i và y_i biểu thị tọa độ ngang và dọc của điểm mốc trên khung hình, đã được chuẩn hóa về khoảng $[0.0, 1.0]$ bằng cách chia cho chiều rộng và chiều cao khung hình; còn z_i biểu thị độ sâu tương đối của điểm mốc đó so với điểm cổ tay (Wrist - điểm số 0), với giá trị z_i càng nhỏ biểu thị điểm khớp càng nằm gần camera hơn.

0.3 Nhận dạng cử chỉ tay bằng Học máy và Heuristics

Hệ thống sử dụng đồng thời hai phương pháp nhận dạng cử chỉ bàn tay nhằm cân bằng giữa tốc độ xử lý và khả năng nhận dạng các mẫu cử chỉ phức tạp.

0.3.1 Phương pháp nhận dạng cử chỉ bằng luật hình học (Heuristics)

Phương pháp này dựa trên trạng thái co hay duỗi của từng ngón tay để xác định các cử chỉ đơn giản mà không cần huấn luyện mô hình. Hệ thống phân tích cấu trúc hình học bàn tay bằng cách tính toán khoảng cách hình học giữa các khớp ngón tay.

Đối với bốn ngón tay dài (ngón trỏ, ngón giữa, ngón áp út và ngón út), mỗi ngón tay f có điểm mốc đầu ngón tay là Tip_f (các điểm 8, 12, 16, 20) và điểm khớp giữa PIP là Pip_f (các điểm 6, 10, 14, 18). Khoảng cách Euclide được sử dụng để đánh giá theo công thức (1):

$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2 + (z_A - z_B)^2} \quad (1)$$

Trạng thái ngón tay f được định nghĩa bằng luật quy định trong phương trình (2):

$$State(f) = \begin{cases} 1 \text{ (duỗi)} & \text{nếu } d(P_{Tip_f}, P_0) > d(P_{Pip_f}, P_0) \\ 0 \text{ (co)} & \text{nếu } d(P_{Tip_f}, P_0) \leq d(P_{Pip_f}, P_0) \end{cases} \quad (2)$$

Trong đó P_0 là tọa độ điểm cổ tay (Wrist).

Đối với ngón cái, hệ thống tính khoảng cách giữa đầu ngón cái (điểm 4) với gốc ngón út (điểm 17) và so sánh với khoảng cách giữa khớp ngón trỏ (điểm 5) và gốc ngón út (điểm 17). Nếu khoảng cách đầu ngón cái vượt quá 1.2 lần khoảng cách tham chiếu khớp ngón trỏ, ngón cái được xác định là duỗi ($State(cái) = 1$), ngược lại là co ($State(cái) = 0$). Tập hợp các trạng thái $[State(cái), State(trỏ), State(giữa), State(áp út), ...]$ cho phép ánh xạ nhanh chóng và chính xác ra các cử chỉ tĩnh cơ bản (Open Hand, Close/Fist, Pointer, Peace) với chi phí tính toán thấp.

0.3.2 Mô hình toán chuẩn hóa tọa độ đặc trưng

Khi sử dụng mô hình học máy để nhận diện nhiều cử chỉ phức tạp, tọa độ thô từ MediaPipe phụ thuộc lớn vào vị trí của bàn tay trong ảnh và khoảng cách của tay tới camera. Để đảm bảo mô hình nhận dạng hoạt động ổn định bất kể tay người dùng ở xa hay gần camera, lệch trái hay lệch phải khung hình, thuật toán chuẩn hóa tọa độ hình học bàn tay được thực hiện như sau:

Bước 1: Tịnh tiến (translation) toàn bộ tọa độ các điểm mốc bàn tay $P_i = (x_i, y_i)$ về gốc tọa độ bằng cách lấy tọa độ điểm cổ tay $P_0 = (x_0, y_0)$ làm điểm mốc chuẩn theo phương trình (3):

$$P'_i = P_i - P_0 = (x_i - x_0, y_i - y_0), \quad \forall i \in [0, 20] \quad (3)$$

Bước 2: Chuyển đổi danh sách 21 điểm mốc P'_i sau tịnh tiến thành một mảng một chiều V gồm 42 phần tử dạng $[x'_0, y'_0, x'_1, y'_1, \dots, x'_{20}, y'_{20}]$. Tìm giá trị tuyệt đối lớn nhất trong mảng một chiều này để xác định hệ số tỉ lệ lớn nhất d_{max} theo công thức (4):

$$d_{max} = \max_{j \in [0, 41]} |V_j| \quad (4)$$

Bước 3: Chia toàn bộ các phần tử trong mảng một chiều cho hệ số tỉ lệ d_{max} để thu được mảng tọa độ chuẩn hóa đặc trưng V'' theo phương trình (5):

$$V''_j = \frac{V_j}{d_{max}}, \quad \forall j \in [0, 41] \quad (5)$$

Mảng đặc trưng V'' gồm 42 phần tử luôn nằm trong khoảng giới hạn $[-1.0, 1.0]$. Phép toán này giảm đáng kể ảnh hưởng của vị trí và kích thước biểu kiến của bàn tay đến camera nhờ việc chuẩn hóa kích thước bàn tay về một tỷ lệ đơn vị thống nhất, tạo điều kiện thuận lợi cho việc huấn luyện mô hình phân loại.

0.3.3 Nhận dạng cử chỉ tĩnh bằng mạng nơ-ron truyền thẳng (MLP)

Để phân loại 6 cử chỉ tĩnh chính bao gồm Open, Close, Pointer, OK, Peace và ThumbsUp, hệ thống sử dụng mạng nơ-ron nhân tạo truyền thẳng nhiều lớp MLP (Multi-Layer Perceptron). Kiến trúc mạng được thiết kế tối ưu với cấu trúc liên kết đầy đủ (Fully Connected) bao gồm một lớp đầu vào nhận vectơ đặc trưng chuẩn hóa V'' gồm 42 chiều, hai lớp ẩn trung gian để học các biểu diễn phi tuyến phức tạp, và một lớp đầu ra dự đoán. Lớp ẩn thứ nhất có 64 nơ-ron được tích hợp kỹ thuật Dropout với tỉ lệ loại bỏ là 0.3 để chống hiện tượng quá khớp (overfitting), và lớp ẩn thứ hai có 32 nơ-ron.

Các lớp ẩn trung gian sử dụng hàm kích hoạt phi tuyến ReLU (Rectified Linear Unit) được định nghĩa qua phương trình (6) nhằm tăng tốc độ hội tụ và tránh hiện tượng triệt tiêu đạo hàm (vanishing gradient):

$$f(x) = \max(0, x) \quad (6)$$

Lớp đầu ra của mạng có 6 nơ-ron tương ứng với 6 lớp cử chỉ tĩnh cần phân loại. Hệ thống áp dụng hàm kích hoạt Softmax ở lớp cuối cùng để chuyển đổi các giá trị đầu ra thô (logits) z_c thành phân phối xác suất dự đoán P cho từng lớp cử chỉ theo phương trình (7):

$$P(class = c|V'') = \frac{e^{z_c}}{\sum_{k=1}^6 e^{z_k}} \quad (7)$$

Mô hình được huấn luyện bằng cách tối ưu hóa hàm mất mát entropy chéo đa lớp dạng thưa (Sparse Categorical Cross-Entropy Loss) được mô tả trong phương trình (8) sử dụng bộ tối ưu hóa Adam:

$$\mathcal{L} = -\log(P(class = y_{true}|V'')) \quad (8)$$

Trong đó y_{true} là chỉ số nhãn thực tế của lớp cử chỉ. Sau khi huấn luyện và lựa chọn bộ trọng số có kết quả tốt trên tập kiểm chứng, mô hình được xuất và chuyển đổi sang định dạng TensorFlow Lite (.tflite). Định dạng TensorFlow Lite (.tflite) giúp mô hình thuận tiện hơn cho quá trình triển khai và suy luận trong các ứng dụng thời gian thực, đồng thời có thể hỗ trợ các kỹ thuật tối ưu hóa dung lượng và tốc độ thực thi trên CPU phổ thông.

0.3.4 Nhận dạng cử chỉ động dựa trên quỹ đạo chuyển động

Nhận dạng cử chỉ động bao gồm các động tác vuốt tay Swipe, vẽ các hình hình học Circle, Triangle, Z-shape đòi hỏi hệ thống phải nắm bắt được thông tin chuyển động theo trục thời gian liên tiếp. Để thực hiện điều này, hệ thống lưu trữ lịch sử

tọa độ không gian của điểm đầu ngón tay trở (landmark số 8) trong một hàng đợi FIFO (First-In, First-Out) có kích thước cố định $N = 32$ khung hình gần nhất.

Để mô hình phân loại cử chỉ động có khả năng hoạt động độc lập với vị trí người dùng vẽ cử chỉ trên màn hình và tốc độ vẽ nhanh hay chậm, chuỗi hành trình tọa độ thô thu được $[(x_0, y_0), (x_1, y_1), \dots, (x_{31}, y_{31})]$ cần được chuẩn hóa. Hệ thống tịnh tiến toàn bộ quỹ đạo về điểm khởi đầu của chuỗi $(x_{start}, y_{start}) = (x_0, y_0)$, đồng thời chia cho kích thước độ phân giải thực tế của khung hình camera (W, H) để khử sự phụ thuộc tỉ lệ theo phương trình (9):

$$x_k^{norm} = \frac{x_k - x_0}{W}, \quad y_k^{norm} = \frac{y_k - y_0}{H}, \quad \forall k \in [0, 31] \quad (9)$$

Mảng phẳng một chiều gồm $32 \times 2 = 64$ phần tử tọa độ chuẩn hóa $[x_0^{norm}, y_0^{norm}, \dots, x_{31}^{norm}, y_{31}^{norm}]$ đại diện cho hình dáng hình học thuần túy của quỹ đạo vẽ. Mảng đặc trưng này được đưa vào mô hình học máy thứ hai là một mạng MLP chuyên biệt dành cho cử chỉ động. Mạng MLP này có lớp đầu vào 64 chiều, được đưa qua một lớp Dropout với tỉ lệ 0.2, tiếp nối bởi lớp ẩn thứ nhất gồm 128 nơ-ron tích hợp Dropout tỉ lệ 0.4, lớp ẩn thứ hai gồm 64 nơ-ron sử dụng kích hoạt ReLU, và lớp đầu ra sử dụng hàm Softmax gồm 10 nơ-ron tương ứng với 10 lớp cử chỉ động.

Sau khi mạng MLP xuất ra kết quả dự đoán thô cho từng khung hình, nhằm triệt tiêu hiện tượng nhảy nhẫ liên tục (label flickering) gây ra bởi nhiễu số từ camera hoặc sai số phân loại tức thời, hệ thống áp dụng cơ chế lọc mượn phi tham số dựa trên biểu quyết đa số (Majority Voting) trong một cửa sổ thời gian trượt. Kỹ thuật này được hiện thực hóa thông qua cấu trúc hàng đợi hai đầu deque với độ dài cố định $M = 32$ phần tử để liên tục tích lũy các nhãn dự đoán thô L_i của các khung hình gần nhất. Nhãn cử chỉ động đầu ra cuối cùng $Class_{final}$ được xác định bằng cách thống kê tần suất và lựa chọn nhãn xuất hiện nhiều nhất trong hàng đợi trượt theo phương trình (10):

$$Class_{final} = \arg \max_{c \in C} \sum_{i=1}^M \mathbb{I}(L_i = c) \quad (10)$$

Trong đó: C là tập hợp 10 lớp cử chỉ động; L_i là nhãn dự đoán thô ở khung hình thứ i trong cửa sổ trượt; và $\mathbb{I}(\cdot)$ là hàm chỉ thị nhận giá trị bằng 1 nếu biểu thức bên trong đúng, nhận giá trị bằng 0 nếu biểu thức sai. Giải pháp làm mượn này giúp ổn định kết quả nhận dạng ở phía Python, tạo cơ sở để tích hợp vào Unity trong các phiên bản tiếp theo, giúp cân bằng hiệu quả giữa tốc độ phản hồi thời gian thực và sự chính xác bền vững của hệ thống.

0.4 Giao thức truyền dữ liệu UDP thời gian thực

Để truyền dữ liệu tọa độ khớp tay và kết quả nhận dạng cử chỉ từ module Python sang môi trường mô phỏng Unity 3D **unity3d**, hệ thống sử dụng giao thức truyền tin qua UDP (User Datagram Protocol) **rfc768udp**.

UDP là giao thức mạng phi kết nối hoạt động ở tầng vận chuyển. So với giao thức TCP đòi hỏi thiết lập kết nối (bắt tay 3 bước) và cơ chế truyền lại khi mất gói tin (gây ra hiện tượng trễ tích lũy cực kỳ nguy hiểm cho các ứng dụng tương tác thời gian thực), UDP cho phép gửi các gói tin một cách độc lập và trực tiếp. Khi dữ liệu được cập nhật liên tục từ camera, việc mất một số gói tin đơn lẻ thường ít ảnh hưởng đến trạng thái hiển thị hơn so với các ứng dụng yêu cầu truyền dữ liệu toàn vẹn, vì gói tin của khung hình tiếp theo có thể nhanh chóng cập nhật trạng thái mới. Do đó, UDP phù hợp với bài toán truyền dữ liệu điều khiển thời gian thực trong phạm vi thử nghiệm cục bộ của đề tài.

0.5 Toán học của bộ lọc thích nghi chống rung One Euro Filter

Tọa độ trích xuất từ camera thường có độ nhiễu tần số cao do rung tay vật lý hoặc sai số từ mô hình MediaPipe. Hiện tượng này làm cho bàn tay ảo hoặc lưỡi dao chém hoa quả trong Unity bị rung nhấp nháy liên tục (jitter). Để giảm hiện tượng này, bộ lọc thông thấp thích nghi One Euro Filter **casiez2012one** được áp dụng trên tọa độ đầu vào của Unity.

One Euro Filter hoạt động dựa trên bộ lọc thông thấp bậc nhất với tần số cắt f_c thay đổi động theo vận tốc của tín hiệu. Cho một tín hiệu đầu vào x_k tại thời điểm t_k , khoảng thời gian trôi qua là $dt = t_k - t_{k-1}$. Các bước tính toán được thực hiện như sau:

Bước 1: Tính đạo hàm rời rạc của tín hiệu đầu vào để ước lượng vận tốc thô dx_k theo công thức (11):

$$dx_k = \frac{x_k - \hat{x}_{k-1}}{dt} \quad (11)$$

Trong đó $\hat{x}_{k-1}$ là giá trị tín hiệu đã được lọc ở khung hình trước.

Bước 2: Lọc thông thấp vận tốc thô dx_k bằng hệ số lọc đạo hàm α_d để thu được vận tốc mượt mà edx_k theo phương trình (12):

$$edx_k = \alpha_d \cdot dx_k + (1 - \alpha_d) \cdot edx_{k-1} \quad (12)$$

Hệ số lọc đạo hàm α_d phụ thuộc vào tần số cắt cố định $f_{c,d}$ (thường chọn mặc định

bằng 1.0 Hz) được xác định theo phương trình (13):

$$\alpha_d = \frac{1}{1 + \frac{\tau_d}{dt}}, \quad \tau_d = \frac{1}{2\pi f_{c,d}} \quad (13)$$

Bước 3: Điều chỉnh tần số cắt thích nghi f_c của tín hiệu chính dựa trên độ lớn vận tốc mượt mà theo công thức (14):

$$f_c = f_{c,min} + \beta \cdot |edx_k| \quad (14)$$

Trong đó: $f_{c,min}$ là tần số cắt tối thiểu (Hz) giúp lọc bỏ tối đa nhiễu rung lắc (jitter) khi tay đứng yên ($|edx_k|$ giảm dần về 0); và β là hệ số nhạy tốc độ giúp tự động tăng tần số cắt f_c khi tay di chuyển nhanh ($|edx_k|$ lớn) để giảm thiểu độ trễ bám đuổi của tín hiệu lọc.

Bước 4: Tính hệ số lọc thích nghi α theo phương trình (15) và giá trị tín hiệu đầu ra sau lọc $\hat{x}_k$ theo phương trình (16):

$$\alpha = \frac{1}{1 + \frac{\tau}{dt}}, \quad \tau = \frac{1}{2\pi f_c} \quad (15)$$

$$\hat{x}_k = \alpha \cdot x_k + (1 - \alpha) \cdot \hat{x}_{k-1} \quad (16)$$

Việc áp dụng One Euro Filter cho cả hai trục tọa độ (x, y) giúp giảm rung lắc khi tay đứng yên, đồng thời vẫn duy trì khả năng bám theo chuyển động nhanh ở mức phù hợp trong quá trình thử nghiệm.

Kết chương

Chương 2 đã trình bày chi tiết toàn bộ các cơ sở lý thuyết toán học và giải thuật được sử dụng trong hệ thống, bao gồm OpenCV tiền xử lý ảnh, MediaPipe Hands trích xuất điểm mốc, các phương trình chuẩn hóa hình học bàn tay, kiến trúc mô hình học máy MLP phân loại cử chỉ tĩnh/động, truyền thông mạng UDP và bộ lọc thích nghi chống rung One Euro Filter. Trên cơ sở lý thuyết này, Chương 3 tiếp theo sẽ trình bày chi tiết về thiết kế kiến trúc hệ thống, cài đặt mã nguồn phần mềm thực tế và đánh giá các kết quả thực nghiệm đạt được.